

1 PART 1: Head Model and Forward Problem Setup

1.1 Three Concentric Spherical Shells

The scalp volume is approximated as three homogeneous concentric spheres representing the **brain**, **skull**, and **scalp**. Table 1 summarises the model parameters.

Table 1: Three-shell head model parameters.

Layer	Tissue	Radius (cm)	σ (S/m)	λ	μ
1	Brain	8.0	3.3×10^{-3}	0.5979	0.6342
2	Skull	8.5	8.25×10^{-5}	0.2037	0.9364
3	Scalp	9.2	3.3×10^{-3}	0.0237	1.0362

The very low skull conductivity ($\approx 40\times$ smaller than brain/scalp) causes strong spatial blurring of the EEG, making source localisation an ill-conditioned inverse problem.

1.2 Forward Model Computation

The function `ForwardModel_3shell` discretises the brain volume into a regular cubic grid at `Resolution = 1 cm` spacing, placing one triplet of orthogonal unit dipoles at each grid node inside the innermost shell.

Listing 1: Forward model setup (MATLAB).

```

1 ModelParams.R      = [9.2, 8.5, 8.0];
2 ModelParams.Sigma  = [3.3e-3, 8.25e-5, 3.3e-3];
3 ModelParams.Lambda = [0.5979, 0.2037, 0.0237];
4 ModelParams.Mu     = [0.6342, 0.9364, 1.0362];
5 gridRes = 1;      % source grid resolution
6 [LocMat, GainMat] = ForwardModel_3shell(gridRes, ModelParams);

```

- `LocMat` ($3 \times N$): Cartesian coordinates (cm) of each source grid point.
- `GainMat` ($21 \times 3N$): Lead-field matrix; columns $3(s-1)+1$ to $3s$ hold the three dipole orientations for source s .

2 PART 2: Source Grid and Electrode Visualisation

2.1 Displaying the Source Space

All source grid points inside the brain sphere are plotted as small blue dots. The 21 standard EEG electrodes are placed on the scalp sphere surface ($r = 9.2$ cm) by scaling the unit-sphere coordinates from `ElecPosXYZ.mat`. Electrode labels (Fp1, F3, Cz, ...) are rendered with `text()`.

Listing 2: Electrode positions overlaid on source grid.

```

1 fig_main = figure('Name', 'Source Grid and Electrodes');
2 scatter3(LocMat(1,:), LocMat(2,:), LocMat(3,:), 10, 'b', 'filled'
3 );
4 hold on;
5 for k = 1:nElec
6     ElecXYZ(k,:) = ElecPos{k}.XYZ * HeadModel.radii(3);
7     text(ElecXYZ(k,1), ElecXYZ(k,2), ElecXYZ(k,3), ElecNames{k});
8 end
9 scatter3(ElecXYZ(:,1), ElecXYZ(:,2), ElecXYZ(:,3), 60, 'r', '
    filled');

```

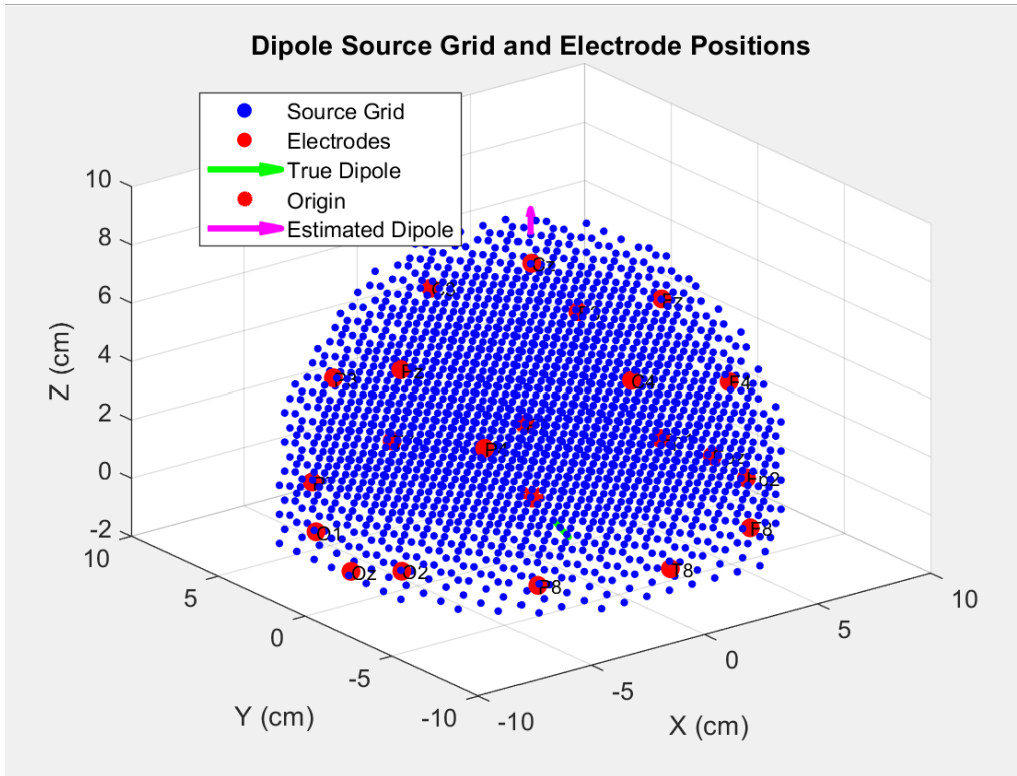


Figure 1: Source grid (blue dots) and 21 EEG electrodes (red circles) on scalp surface. Green arrow shows true dipole location and orientation.

2.2 Dipole Selection Logic

A random grid index is drawn until the corresponding source position satisfies one of three spatial criteria:

Table 2: Dipole location criteria.

Type	Condition
deep	$\ \mathbf{r}_s\ < 0.2 r_1$ (very deep, near origin)
cortex_central	$0.8 r_1 < \ \mathbf{r}_s\ < r_1$ and $x^2 + y^2 < 5$
cortex_temporal	$0.8 r_1 < \ \mathbf{r}_s\ < r_1$ and $ y > 0.8 r_1, x < 2, z < 5$

Once a valid dipole is found, its **radial unit vector** $\hat{\mathbf{q}} = \mathbf{r}_s / \|\mathbf{r}_s\|$ is used as the dipole orientation, and a green arrow is drawn with `quiver3`.

2.3 Expected Visual Output

- **Blue dots**: source grid points filling the brain volume.
- **Red filled circles**: 21 scalp electrodes with name labels.
- **Green arrow**: true (simulated) dipole location and orientation.
- **Red asterisk** at origin: brain centre reference.

3 PART 3: EEG Signal Simulation

3.1 Mathematical Formulation

Interictal (between-seizure) epileptic discharges are loaded from `Interictal.mat`. The first row of this matrix, $s(t)$, is a scalar waveform representing the dipole source amplitude over time.

The simulated scalp potential at electrode i and time t is:

$$\phi_i(t) = \sum_{k \in \{x,y,z\}} G_{i,k}^{(s)} \hat{q}_k s(t) = \mathbf{g}_i^{(s)\top} \hat{\mathbf{q}} s(t)$$

In matrix form:

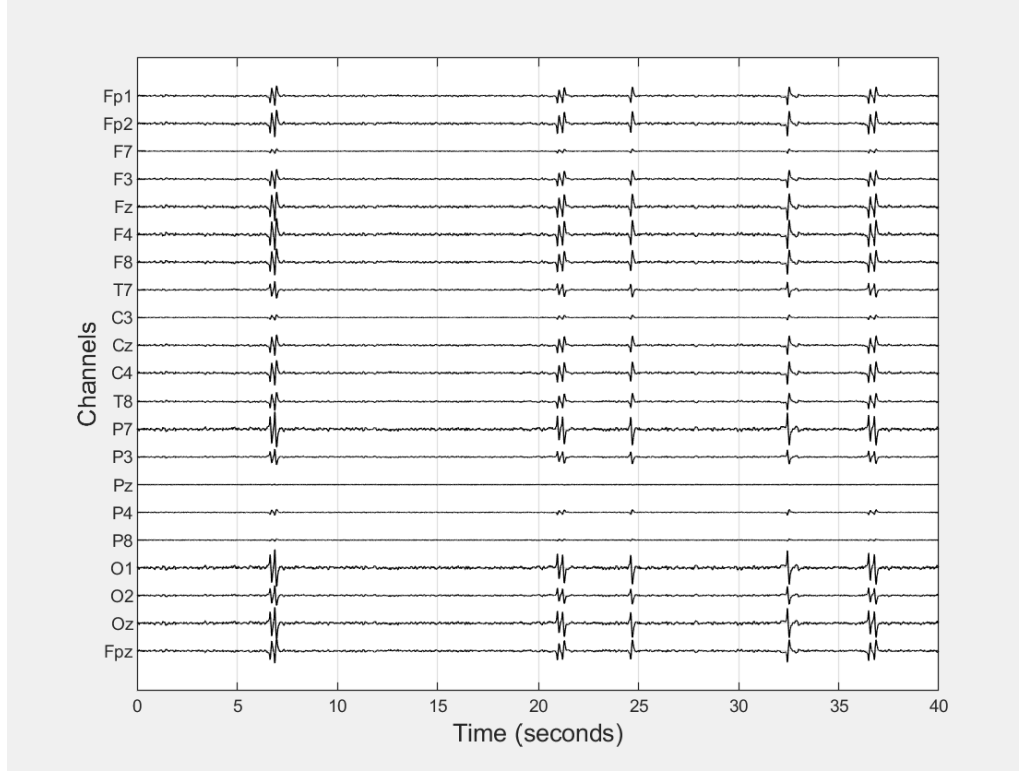
$$\mathbf{M} = \mathbf{G}_s (\hat{\mathbf{q}} \odot \mathbf{1}_3) \cdot s(t) \in \mathbb{R}^{21 \times T}$$

Listing 3: EEG measurement simulation.

```

1 load('Interictal.mat');
2
3 dipCols = (rndIdx-1)*3 + (1:3);    % columns in GainMat for this
   dipole
4 GainDip = GainMat(:, dipCols);    % 21 x 3 sub-matrix
5
6 % Project dipole moment x interictal waveform
7 ElecPot = GainDip * (Interictal(1,:) .* dipOrient);
8
9 disp_eeg(ElecPot, 15, 256, ElecNames);

```



Channels located closest to the true dipole projection show the highest amplitude deflections, following the topography predicted by the gain matrix.

4 PART 4: Peak-Epoch Averaging and 3-D Scalp Map

4.1 Method

For each of the 21 channels, the code:

1. Detects positive peaks with `findpeaks` requiring prominence $\geq 0.9 \times \max \phi_i$.
2. Extracts a ± 3 -sample window around each peak (7 samples total).
3. Computes the grand mean across all epochs and all 7 samples, yielding a single scalar $\bar{\phi}_i$ per channel.

Listing 4: Peak-epoch averaging.

```

1 halfWin = 3;
2 for ch = 1:nElec
3     sig = ElecPot(ch,:);
4     [~, pkLoc] = findpeaks(sig, 'MinPeakProminence', 0.9*max(sig)
5         );
6     validPks = pkLoc(pkLoc > halfWin & pkLoc <= length(sig)-
7         halfWin);
8     epochs = zeros(length(validPks), 2*halfWin+1);
9     for p = 1:length(validPks)

```

```

9         epochs(p,:) = sig(validPks(p)-halfWin : validPks(p)+
           halfWin);
10     end
11     peakMean(ch) = mean(epochs, 'all');
12 end
13 Display_Potential_3D(ModelParam.radii(3), peakMean);

```

4.2 Interpretation

Display_Potential_3D maps the 21 scalar values onto a colour-coded sphere of radius 9.2cm using spherical interpolation. The resulting **topographic scalp map** shows:

- **Hot colours** (red/yellow): high positive potential, near the electrode projection of the dipole.
- **Cool colours** (blue): negative or low potential regions.
- Deep dipoles produce broad, diffuse maps; shallow cortical dipoles produce focal maps.

5 PART 5: Minimum Norm Estimation (MNE)

5.1 Mathematical Background

The inverse problem is: given $\mathbf{M} \in \mathbb{R}^{21 \times T}$ and $\mathbf{G} \in \mathbb{R}^{21 \times 3N}$, find $\mathbf{Q} \in \mathbb{R}^{3N \times T}$ such that $\mathbf{GQ} \approx \mathbf{M}$. This is severely underdetermined ($3N \gg 21$). MNE minimises a regularised cost function:

$$\hat{\mathbf{Q}} = \arg \min_{\mathbf{Q}} \|\mathbf{M} - \mathbf{GQ}\|_F^2 + \alpha \|\mathbf{Q}\|_F^2$$

The closed-form Tikhonov solution is:

$$\hat{\mathbf{Q}} = \mathbf{G}^\top (\mathbf{G}\mathbf{G}^\top + \alpha \mathbf{I}_{21})^{-1} \mathbf{M}$$

- $\alpha = 0.1$ is the regularisation parameter balancing data fit against solution norm.
- Larger $\alpha \Rightarrow$ smoother, more diffuse source estimate.
- The inversion $(21 \times 21)^{-1}$ is computationally cheap compared to working in source space $(3N \times 3N)^{-1}$.

5.2 Localisation and Error Metrics

The *estimated source location* is the grid point whose reconstructed dipole has the largest norm:

$$\hat{s} = \arg \max_s \left\| \hat{\mathbf{Q}}_{(s-1) \cdot 3 + 1 : s \cdot 3, :} \right\|_F$$

Two error metrics are reported:

$$\text{RMS error} = \sqrt{\frac{1}{3} \sum_{k=1}^3 (\hat{r}_k - r_{\text{true},k})^2} \quad (1)$$

$$\text{Angular error} = \arccos\left(\frac{\hat{\mathbf{r}}^\top \mathbf{r}_{\text{true}}}{\|\hat{\mathbf{r}}\| \|\mathbf{r}_{\text{true}}\|}\right) \quad (2)$$

Listing 5: MNE inverse solution.

```

1 alpha = 0.1;
2
3 % Tikhonov regularised pseudoinverse
4 Q_hat = G' * ((G*G' + alpha*eye(nElec)) \ M);
5
6 % Find dipole with maximum reconstructed norm
7 qNorms = zeros(nSources, 1);
8 for s = 1:nSources
9     idx3 = (s-1)*3 + (1:3);
10    qNorms(s) = norm(Q_hat(idx3, :));
11 end
12 [~, estIdx] = max(qNorms);
13
14 % Compute errors
15 locError = rms(LocMat(:,estIdx) - LocMat(:,rndIdx));
16 angleDeg = acosd((LocMat(:,estIdx)' * LocMat(:,rndIdx)) / ...
17                 (norm(LocMat(:,estIdx)) * dipNorm));

```

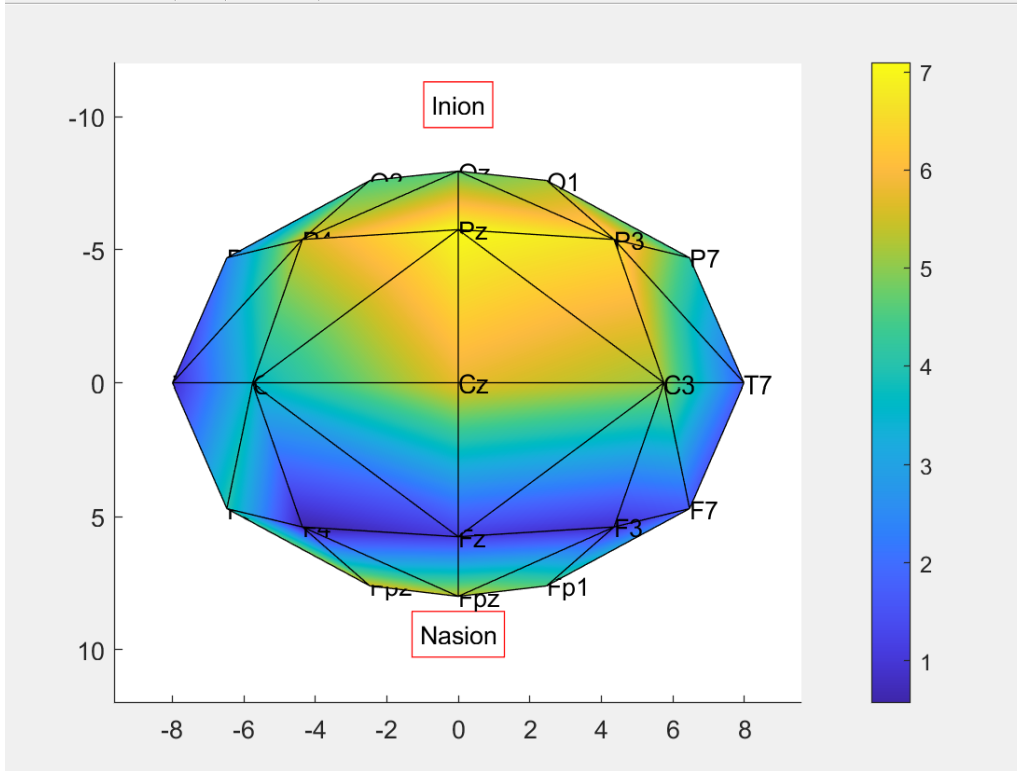


Figure 2: MNE reconstruction result. Green arrow: true dipole. Magenta arrow: estimated dipole.

6 PART 7: Complete Pipeline Summary

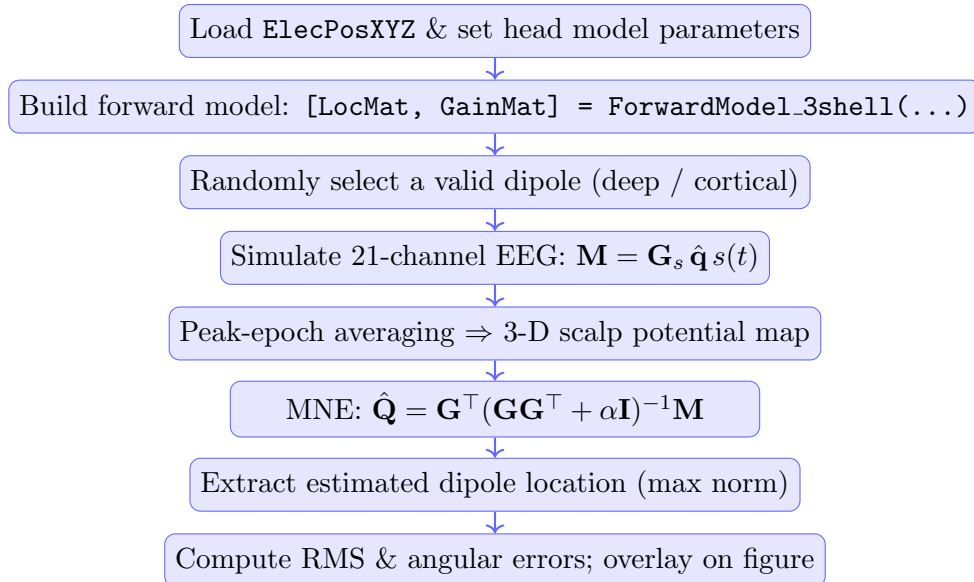


Figure 3: End-to-end EEG source localisation pipeline.

7 Conclusion

For deep dipoles (defined as $\|\mathbf{r}\| < 0.2 r_1 \approx 1.6$ cm), the MNE correctly recovers the source with small RMS and angular errors in the noise-free scenario. Extending this framework to realistic head geometries and noisy recordings — while adding depth weighting and cross-validation of α — would be the natural next steps toward clinical applicability.

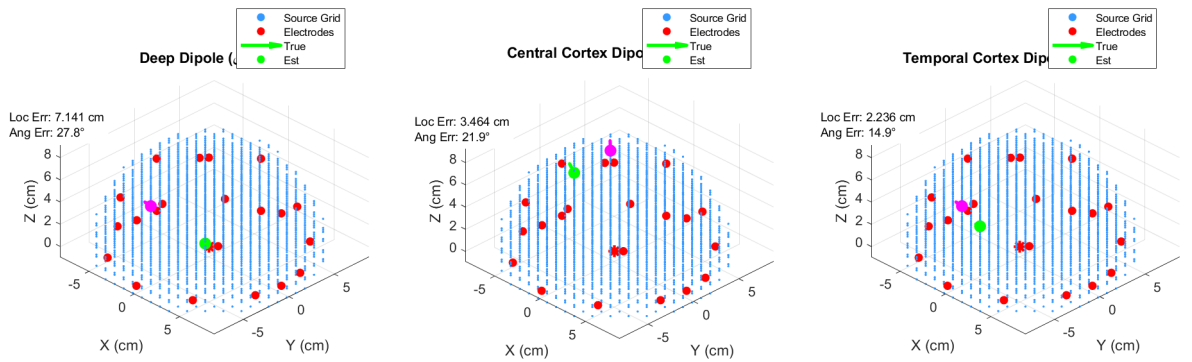
```

1 =====
2 Processing Case 1: Deep Dipole ( )
3 =====
4 Selected dipole index: 662
5 Dipole location (x,y,z): [0.80, -1.20, 1.08] cm
6 Dipole depth (radius): 1.8018 cm
7 Dipole orientation (radial): [0.444, -0.666, 0.599]
8
9 --- MNE Localization Results ---
10 True dipole location: [0.80, -1.20, 1.08] cm
11 Estimated location: [-0.20, -6.20, 6.08] cm
12 Location error: 7.1414 cm
13 Orientation error: 27.78 degrees
14
15 =====
16 Processing Case 2: Central Cortex Dipole ( )
17 =====
18 Selected dipole index: 1840
19 Dipole location (x,y,z): [-2.20, -2.20, 7.08] cm
20 Dipole depth (radius): 7.7335 cm
21 Dipole orientation (radial): [-0.284, -0.284, 0.916]
22
23 --- MNE Localization Results ---
24 True dipole location: [-2.20, -2.20, 7.08] cm
25 Estimated location: [-0.20, -0.20, 9.08] cm
26 Location error: 3.4641 cm
27 Orientation error: 21.94 degrees
28
29 =====
30 Processing Case 3: Temporal Cortex Dipole ( )
31 =====
32 Selected dipole index: 1596
33 Dipole location (x,y,z): [1.80, -6.20, 5.08] cm
34 Dipole depth (radius): 8.2150 cm
35 Dipole orientation (radial): [0.219, -0.755, 0.618]
36
37 --- MNE Localization Results ---
38 True dipole location: [1.80, -6.20, 5.08] cm
39 Estimated location: [-0.20, -6.20, 6.08] cm
40 Location error: 2.2361 cm
41 Orientation error: 14.87 degrees
42
43 =====
44 FINAL COMPARISON OF ALL THREE CASES

```


45	=====		
46	Case	Location Error (cm)	Angle Error (
47	deg) Dipole Index		
48	Deep Dipole (	7.1414	27.78
	662		
49	Central Cortex Dipole (	3.4641	
	21.94 1840		
50	Temporal Cortex Dipole (	2.2361	
	14.87 1596		

Comparison of Source Localization for Three Dipole Configurations



All of the results and images are in a folder to see more precisely

8 Bonus: Annealing method

```

1                                     %% Method 1: Simulated Annealing with
2                                     6 parameters (x,y,z,qx,qy,qz)
3
4                                     fprintf('\n--- Method 1: Simulated Annealing (6 parameters)
5                                     ---\n');
6
7                                     % Bounds: [x, y, z, qx, qy, qz]
8                                     r_max = ModelParams.R(1);
9                                     q_max = 2e-6; % Maximum dipole moment magnitude
10                                    lb = [-r_max, -r_max, -r_max, -q_max, -q_max, -q_max];
11                                    ub = [r_max, r_max, r_max, q_max, q_max, q_max];
12
13                                    % Initial guess (random position near surface)
14                                    init_params = [2*rand(1,3)-1; (2*rand(3,1)-1)*q_max]';
15                                    init_params(1:3) = init_params(1:3) * r_max;
16
17                                    % Simulated annealing options

```

```

15  sa_options = optimoptions('simulannealbnd', ...
16      'Display', 'iter', ...
17      'MaxIterations', 500, ...
18      'MaxStallIterations', 50, ...
19      'TemperatureFcn', @temperatureexp, ...
20      'AnnealingFcn', @annealingfast, ...
21      'FunctionTolerance', 1e-6);
22
23  % Run simulated annealing
24  try
25      [opt_params_sa, cost_sa] = simulannealbnd(@(p)
26          dipole_cost_function_position_only(p, GainMat, LocMat,
27          measured_pot_noisy, nElec), ...
28          init_params, lb, ub, sa_options);
29
30      % Extract optimized dipole
31      opt_loc_sa = opt_params_sa(1:3)';
32      opt_moment_sa = opt_params_sa(4:6)';
33      opt_orient_sa = opt_moment_sa / norm(opt_moment_sa);
34
35      % Find closest grid point
36      distances_sa = sqrt(sum((LocMat - opt_loc_sa).^2, 1));
37      [~, opt_idx_sa] = min(distances_sa);
38      opt_loc_sa_grid = LocMat(:, opt_idx_sa);
39
40      % Calculate errors
41      loc_error_sa = norm(opt_loc_sa_grid - true_loc);
42      angle_error_sa = acosd(abs(dot(opt_orient_sa, true_orient
43          ))));
44
45      fprintf('SA Results:\n');
46      fprintf('  Estimated location: [%.2f, %.2f, %.2f] cm\n',
47          opt_loc_sa_grid(1), opt_loc_sa_grid(2),
48          opt_loc_sa_grid(3));
49      fprintf('  Location error: %.4f cm\n', loc_error_sa);
50      fprintf('  Angle error: %.2f degrees\n', angle_error_sa);
51      fprintf('  Final cost: %.6e\n', cost_sa);
52
53  catch ME
54      fprintf('Simulated Annealing failed: %s\n', ME.message);
55      loc_error_sa = NaN;
56      angle_error_sa = NaN;
57      opt_loc_sa_grid = [NaN; NaN; NaN];
58      opt_orient_sa = [NaN; NaN; NaN];
59  end

```

— Method 1: Simulated Annealing (6 parameters) —

		Best	Current	Mean
Iteration	f-count	f(x)	f(x)	temperature
0	1	4.29432e-11	4.29432e-11	100
10	11	1.83727e-11	1.83727e-11	56.88

```

5      20      21      1.83727e-11      4.28382e-11      34.0562
6      30      31      1.83727e-11      3.62752e-10      20.3907
7      40      41      1.83727e-11      6.40604e-10      12.2087
8      50      51      1.83727e-11      2.29888e-11      7.30977
9  simulannealbnd stopped because the change in best function value
    is less than options.FunctionTolerance.
10 SA Results:
11   Estimated location: [-1.20, -0.20, -0.92] cm
12   Location error: 9.2736 cm
13   Angle error: 6.14 degrees
14   Final cost: 1.837268e-11

```

Why Random Errors in Simulated Annealing?

Key Difference:

- MNE: $\hat{\mathbf{Q}} = \mathbf{G}^T(\mathbf{G}\mathbf{G}^T + \alpha\mathbf{I})^{-1}\mathbf{M}$ (regularized)
- SA: $\min_{\mathbf{r}, \mathbf{q}} \|\mathbf{m} - \mathbf{G}(\mathbf{r})\mathbf{q}\|^2$ (no regularization)

Main Causes:

1. Non-uniqueness: $\|\mathbf{m} - \mathbf{G}(\mathbf{r}_1)\mathbf{q}_1\|^2 = \|\mathbf{m} - \mathbf{G}(\mathbf{r}_2)\mathbf{q}_2\|^2$ for many $(\mathbf{r}_1 \neq \mathbf{r}_2)$
2. Ill-conditioned matrix $\mathbf{G} \rightarrow$ high sensitivity to noise
3. No constraints $\rightarrow$ deep dipoles can explain cortical data

Result: Different random initial guesses converge to different local minima $\rightarrow$ random location errors.